SYSTEM ARCHITECTURE & ENGINEERING SPECIFICATION

# Tri-Layer Expense Management Architecture

A Deterministic Engine Integrating Zero-Based Envelopes, Cash Flow Projections, and Dynamic Safe-to-Spend Limits

## 1. Executive Summary & Foundational Model

Traditional personal budgeting solutions routinely fail due to two fundamental architectural deficiencies: retrospective latency (logging costs after capital has departed) and cognitive burnout (requiring manual transaction reconciliation across dozens of granular categories). This application introduces a unified three-layer financial operating engine that resolves these challenges through discrete computational abstractions.

| Layer | Core Responsibility | Mathematical Mechanism | User Cognitive Footprint |
|---|---|---|---|
| **Layer 1: Allocation** | Zero-Based Virtual Envelopes | Priority-tier waterfall algorithm with strict zero-sum constraint. | Zero manual transfers; paychecks auto-split into deterministic buckets. |
| **Layer 2: Engine** | Cash Flow Timeline | 60-day discrete-time recursive bank projection and solvency verification. | Forward-looking timeline graph warning of impending shortfalls weeks in advance. |
| **Layer 3: Interface** | Dynamic Safe-to-Spend | Discretionary pool divided by remaining cycle days with smoothed overspend dampening. | Single actionable metric: real-time daily available discretionary spend. |

## 2. Layer 1: Zero-Based Waterfall Allocation Engine

The allocation engine governs the logical assignment of liquid capital into virtual compartments without altering physical bank accounts. Let the set of envelopes $E$ be partitioned into three disjoint subsets: Fixed Contractual Obligations ($E_{fixed}$), Sinking Funds ($E_{sink}$), and Discretionary Pools ($E_{disc}$).

### CORE INVARIANCE RULE & INFLOW ALLOCATION

For any income inflow event $I(t)$ at day $t$, 100% of capital must be assigned:

$$I(t) = \sum_{j \in E} A_j(t)$$

where $A_j(t) \geq 0$ represents the virtual allocation assigned to envelope $j$.

### Waterfall Execution Hierarchy

1. **Priority Tier 1 (Fixed Contractual):** Fully funds fixed recurring obligations due before the subsequent income cycle: $A_j(t) = min(Remaining\ Inflow, DueAmount_j - B_j(t))$.
2. **Priority Tier 2 (Sinking Target Caps):** Dispatches scheduled amortized contributions toward long-range irregular targets: $A_j(t) = min(Remaining\ Inflow, Installment_j)$ subject to $B_j(t) \le Cap_j$.
3. **Priority Tier 3 (Discretionary Residual):** Any remaining capital after Tiers 1 and 2 are fully satisfied flows into the active cycle discretionary pool: $A_{disc}(t) = max(0, Remaining\ Inflow)$.

### Envelope Rollover and Deficit Policies

- **Accumulation Policy (Sinking Funds):** Balance $B_j(t)$ rolls over unconditionally across monthly boundaries to compound toward the defined cap.
- **Top-Up Policy (Fixed Utilities):** Variable fixed bills (e.g., electricity) retain leftover funds; the subsequent cycle only assigns the marginal delta up to the expected ceiling.
- **Reset & Sweep (Discretionary):** Leftover discretionary funds at the end of a pay cycle are swept into Tier 2 investment or emergency reserve envelopes, establishing a clean slate for the next cycle.
- **Deficit Absorption Hierarchy:** If an envelope incurs an unexpected overdraw, the system automatically absorbs the delta directly from $B_{disc}$, dynamically reducing future Safe-to-Spend without corrupting fixed reserves.

## 3. Layer 2: Cash Flow & Burn-Rate Forecast Engine

While Layer 1 organizes virtual capital, Layer 2 models physical solvency across time $t \in [0, H]$ over a rolling forward horizon $H = 60$ days.

> **TEMPORAL BALANCE RECURSION & SOLVENCY CONSTRAINT**
>
> $$C(\tau) = C(\tau - 1) + I(\tau) - O_{sched}(\tau) - S_{daily}(cycle(\tau))$$
>
> Solvency Condition: $\min_{0 \le \tau \le H} C(\tau) \ge C_{min}$
>
> Warning Trigger Band: $C(\tau) < C_{min} + (1.5 \times S_{daily})$

Here $C(\tau)$ denotes aggregate bank account liquidity on day $\tau$, $I(\tau)$ represents scheduled income events, $O_{sched}(\tau)$ encompasses scheduled fixed debits, and $C_{min}$ is the non-negotiable liquidity reserve floor (e.g., $200.00).

### Recurrence Rules & Business Day Normalization

Events are indexed through standardized calendar recurrence vectors (Weekly, Bi-Weekly, Semi-Monthly, Monthly, and Annual). To mirror actual banking behavior:

- **Inflows (Direct Deposits):** Falling on a weekend or public holiday roll backward to the preceding Friday (earlier liquidity).
- **Outflows (Automated Debits):** Falling on non-business days roll forward to the next business day (delayed debit).

## 4. Layer 3: Dynamic Safe-to-Spend Computation

Layer 3 distills multi-variable financial complexity into an intuitive daily spending ceiling. This metric eliminates transaction categorization anxiety by treating all flexible daily purchases as withdrawals against a unified daily allowance.

> **DAILY ALLOWANCE & DAMPENED OVERSPEND RECALIBRATION**
>
> Base Daily Rate: $S_{daily}(t) = \max(0, B_{disc}(t) - O_{unfunded}) / N_p$
>
> Effective Allowance Today: $S_{today}(t) = S_{daily}(t) - E_{disc_actual}(t)$
>
> Recalibration on Day t+1: $S_{daily}(t + 1) = B_{disc}(t + 1) / (N_p - 1)$

Where $N_p$ is the count of calendar days remaining until the next scheduled income deposit, and $O_{unfunded}$ denotes upcoming contractual liabilities lacking envelope backing. When actual spending exceeds $S_{today}(t)$, the engine does not issue punitive alerts; it distributes the variance across all remaining days in the cycle, incrementally lowering the daily spending target.

## 5. Relational Data Architecture

The application data schema maintains clean boundaries between physical accounts, logical envelopes, recurring event schedules, and ledger entries.

```
-- Core Relational Schema (PostgreSQL) CREATE TABLE accounts ( id UUID PRIMARY KEY
DEFAULT gen_random_uuid(), name VARCHAR(100) NOT NULL, current_balance NUMERIC(12, 2)
NOT NULL DEFAULT 0.00, updated_at TIMESTAMPTZ DEFAULT NOW() ); CREATE TABLE envelopes (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(), name VARCHAR(100) NOT NULL,
tier_category VARCHAR(20) NOT NULL CHECK (tier_category IN ('fixed', 'sinking',
'discretionary')), priority_order INT NOT NULL, current_balance NUMERIC(12, 2) NOT NULL
DEFAULT 0.00, target_cap NUMERIC(12, 2) NULL, monthly_installment NUMERIC(12, 2) NULL,
rollover_policy VARCHAR(20) NOT NULL CHECK (rollover_policy IN ('accumulate', 'top_up',
'reset_sweep')) ); CREATE TABLE scheduled_schedules ( id UUID PRIMARY KEY DEFAULT
gen_random_uuid(), envelope_id UUID REFERENCES envelopes(id) ON DELETE SET NULL,
account_id UUID NOT NULL REFERENCES accounts(id) ON DELETE CASCADE, title VARCHAR(120)
NOT NULL, amount NUMERIC(12, 2) NOT NULL, direction VARCHAR(10) NOT NULL CHECK
(direction IN ('inflow', 'outflow')), recurrence_rrule VARCHAR(100) NOT NULL,
next_due_date DATE NOT NULL, is_active BOOLEAN DEFAULT TRUE ); CREATE TABLE
transactions ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), account_id UUID NOT NULL
REFERENCES accounts(id) ON DELETE CASCADE, envelope_id UUID REFERENCES envelopes(id) ON
DELETE RESTRICT, amount NUMERIC(12, 2) NOT NULL, booked_date DATE NOT NULL, memo TEXT
);
```

## 6. Multi-Tenancy Architecture & Data Isolation

To cleanly support solo users, shared domestic partnerships, complex households, and financial advisory engagements, the platform implements a pooled single-database, shared-schema tenancy model fortified by native PostgreSQL Row-Level Security (RLS). Every state-bearing entity maintains an indexed foreign key referencing `tenants(id)`.

```
-- Tenancy and Access Control Schema CREATE TABLE tenants ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), name VARCHAR(100) NOT NULL, tenant_type VARCHAR(20) NOT NULL CHECK (tenant_type IN ('individual', 'household', 'organization')), base_currency CHAR(3) NOT NULL DEFAULT 'USD', created_at TIMESTAMPTZ DEFAULT NOW() ); CREATE TABLE tenant_memberships ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE, user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, role VARCHAR(30) NOT NULL CHECK (role IN ('owner', 'admin', 'member', 'read_only', 'advisor')), status VARCHAR(20) NOT NULL DEFAULT 'active' CHECK (status IN ('invited', 'active', 'suspended')), created_at TIMESTAMPTZ DEFAULT NOW(), UNIQUE (tenant_id, user_id) ); -- Granular Sub-Envelope Visibility Overrides CREATE TABLE envelope_access_overrides ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), envelope_id UUID NOT NULL REFERENCES envelopes(id) ON DELETE CASCADE, user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, permission_level VARCHAR(20) NOT NULL CHECK (permission_level IN ('full', 'spend_only', 'hidden')), UNIQUE (envelope_id, user_id) );
```

PostgreSQL session context variables set by the authentication middleware (`app.current_tenant_id` and `app.current_user_id`) enforce isolation at the database engine tier, mitigating accidental cross-tenant data leakage in application queries.

## 7. Role-Based Access Control (RBAC) Matrix

The system establishes five explicit role profiles governing interaction with core financial machinery:

| Permission / Capability | Owner | Admin | Member | Advisor | Read-Only |
|---|---|---|---|---|---|
| Manage Tenant, Billing & Ownership | Allowed | Denied | Denied | Denied | Denied |
| Invite / Revoke Users & Assign Roles | Allowed | Allowed | Denied | Denied | Denied |
| Connect / Modify Physical Accounts | Allowed | Allowed | Denied | Denied | Denied |
| Configure Envelopes & Waterfall Rules | Allowed | Allowed | Denied | Propose Only | Denied |
| Log Daily Discretionary Spend | Allowed | Allowed | Allowed | Denied | Denied |
| Inspect Safe-to-Spend & Timeline Forecast | Allowed | Allowed | Allowed | Allowed | Allowed |

## 8. Daily Evaluation & Execution Pipeline

The state machine executes synchronously upon each recorded transaction and runs as a batch service every midnight:

1. **Tenant Context Initialization:** Establishes strict tenant boundary security context prior to ledger manipulation.
2. **Ledger Synchronization:** Updates physical bank balances across all active tenant accounts.
3. **Envelope Settlement:** Applies waterfall allocations to incoming credits; deducts registered expenditures from designated envelope balances.
4. **Forward Simulation:** Projects physical balance $C(\tau)$ across the 60-day horizon using active recurrence rules and expected daily discretionary burn rate.
5. **Solvency Check:** Evaluates if $C(\tau) < C_{min}$ on any future date. If violated, an alert generates actionable mitigation steps (e.g., dynamic daily ceiling reduction).
6. **Safe-to-Spend Calibration:** Computes the new $S_{daily}$ for distribution to client applications.